

Software Architecture in the Age of Agentic AI

Karthik Vaidhyanathan[1(✉)] and Henry Muccini[2]

[1] SERC, IIIT-Hyderabad, Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in
[2] FrAmeLab, University of L'Aquila, L'Aquila, Italy
henry.muccini@univaq.it

Abstract. The increasing use of generative AI and agent-based systems in software development has led to new challenges and opportunities in software architecture. Two broad areas of research are emerging: i) software architecture (SA) for building AI-enabled systems, including those with LLMs and autonomous agents; and ii) the use of AI techniques, especially generative models, to support architectural activities such as decision-making, documentation, etc. While both directions are gaining interest, contributions are scattered across different research communities including software architecture, AI/ML, and self-adaptive systems.

This half-day tutorial aims to bring together ideas and experiences from both research and practice in a structured way. It is divided into three parts: i) a brief introduction to SA for ML-enabled systems (SAML), ii) LLMs for supporting software architectural processes and practices (LLM4SA), and iii) Architecting LLM-based multi-agent systems (Agentic AI).

Each part will include key terminology, research themes, example systems/demos/case studies, and hands-on/discussion activities. The goal is to help participants understand the current state of research, share practical tools and frameworks, and identify future research directions in this fast-evolving area.

Keywords: Software Architecture · Agentic AI · Generative AI · Large Language Models · Machine Learning · LLM

1 Tutorial Description

The rapid advancements in generative AI—especially large language models (LLMs)—has introduced new possibilities in how software systems are designed, developed, and evolved [6,9,10]. These systems are no longer static or rule-driven but are increasingly dynamic, learning-enabled, and often composed of intelligent components capable of autonomous decision-making. At the same time, the growing levels of abstraction also implies that the role of software architecture as well as architects (as a role) is becoming more critical than ever.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli et al. (Eds.): ECSA 2025, LNCS 15982, pp. 41–49, 2026.
https://doi.org/10.1007/978-3-032-04403-7_5

Architecting modern AI-enabled software systems is not just about considering traditional quality attributes like performance, maintainabililty etc. but it is also about considering other key quality attributes like observability, uncertainty, explainability, trustworthiness, data eccentricity, evolovability of agentic behaviour, etc. [7].

This shift has led to two parallel and fast-growing areas of interest: (i) software architecture for AI-enabled systems—where traditional architectural methods are being extended to handle the complexities of ML, LLMs, and multi-agent intelligence [8,11]; and (ii) the use of AI, particularly generative models, to support software architecture practice itself—through automation, design assistance, and decision support [1–4]. While there are works emerging in both directions, the research and tooling are fragmented across different communities. Moreover, there is also a need to create awareness in these emerging areas among the software architecture research and practitioner communities.

To this end, this tutorial aims to explore the growing connection between software architecture and AI, with a dual focus: i) On one hand, we look at how to architect systems that include AI components, such as machine learning models, large language models (LLMs), or AI agents. This is about adapting and extending software architecture practices to handle the complexity, adaptability, and unique challenges of AI-enabled systems [11]; ii) On the other hand, we also explore how AI itself can help with software architecture—for example, using generative AI to support architectural decision-making, automate documentation, or guide design trade-offs.

Covered Topics. While both of the directions of using AI for software architecture and Architecting AI-enabled systems are becoming active research areas, they are still developing in parallel, often across different research communities. This tutorial brings them together in a single, structured session that blends research insights with practical examples and hands-on activities.

The tutorial is divided into three connected parts:

Part 1: Software Architecture for ML-enabled Systems (SAML)
In this part, we look at how traditional software architecture needs to change when systems include ML components. Unlike typical software, ML-enabled systems require handling data pipelines, training workflows, inference pipelines, and feedback loops. This part will provide the foundations of software architecture for ML-enabled systems by setting some background and context.
This segment will help participants build a solid foundation in thinking architecturally about ML-enabled systems.

Part 2: LLM for Software Architecture (LLM4SA)
This part focuses on the reverse direction: how GenerativeAI, and specifically LLMs, can assist software architects. With recent advances, LLMs are now being used as co-pilots that help in writing architectural documentation, generating ADRs, analyzing trade-offs, and even recommending patterns. In this part, the tutorial will:

- Introduce terminology such as prompts, context windows, and model chaining.
- Explore practical use cases where LLMs are embedded into architecture workflows.
- Discuss how to evaluate and validate the outputs of LLMs in this context.
- Include a short hands-on session using an open source tool developed by one of the tutorial presenters based on the work done by Dhar et al. [2].

The goal is to help participants see the potential of using AI to support architectural thinking and not just system functionality.

Part 3: Architecting Agentic AI Systems (Agentic AI)
This final part focuses on agentic AI systems—those made up of multiple autonomous AI agents that can plan, reason, and work together. These systems require a different way of thinking about architecture, especially when it comes to coordination, communication, and adaptability. This part of the tutorial will introduce:

- The concept of agentic systems using examples from current research and practice.
- Show how multi-agent systems (MAS) are evolving with the rise of LLMs and orchestration frameworks (like AutoGen, CrewAI, Google ADK, etc.).
- Explain how architecture supports autonomy, memory, message passing, and dynamic decision-making.
- Include a hands-on case study where participants design the architecture of a simple multi-agent system (e.g., for autonomous CloudOps using MOYA [11]).

This part brings together many of the earlier ideas and prepares participants to think systemically about the future of architecting Agentic AI systems.

1.1 Learning Objectives

This tutorial is designed to help participants build both foundational understanding and practical skills at the intersection of software architecture and Generative AI. Using Bloom's Taxonomy as a reference [5], the following learning objectives are addressed:

1. Recall key terminology and concepts related to AI-enabled software systems.
2. Understand how LLMs can act as co-pilots in architecture-related tasks like architecture design decisions.
3. Evaluate the fitness of architectural decisions produced by LLMs for given scenarios.
4. Critically assess the role of LLMs in software architecture practices—considering reliability, reproducibility, explainability, and sustainability.
5. Analyze the architectural implications of including LLMs or agentic AI components within an existing software system.
6. Design a multi-agent architecture for a real-world case study, integrating generative AI components.

1.2 Key Takeaways for the Audience

This tutorial aims to provide both conceptual understanding and hands-on skills. Below are the key takeaways that participants can expect:

- **A clear understanding of the dual role of AI and Software Architecture**
  Participants will gain a solid understanding of both directions:
  - Architecting AI-enabled systems, including ML, LLMs, and agentic systems.
  - Using AI (especially LLMs) to support software architecture practices.
- **Familiarity with ML-specific architectural concepts and patterns**
  Participants will learn about:
  - ML pipeline components and their architectural implications.
  - Architecting multi-agent systems with coordination and autonomy.
- **Hands-on experience with ML-enhanced architecture design**

  Through interactive sessions, participants will:
  - Use an existing LLM based co-pilot to generate architecture design decisions.
  - Design a multi-agent architecture for a real-world case study.
- **A shared vocabulary across SA and LLM communities**

  The tutorial will help participants bridge terminology gaps by introducing:
  - Key terms such as prompts, chaining, Agentic AI, orchestration, memory, agentic AI protocols like MCP, A2A, etc. and tool calling.
  - Concepts that promote interdisciplinary communication.
- **Awareness of current tools, frameworks, and practices**

  Participants will be introduced to:
  - Agent orchestration frameworks like CrewAI, MOYA, Google ADK, etc.
  - LLM-based co-pilot tools for architecture support.
  - References to datasets, benchmarks, and research tools.
- **Exposure to open research questions and challenges**

  The tutorial will highlight:
  - Key gaps in AI-for-software architecture and software architecture-for-AI research.
  - Challenges in using LLMs for software architecture specific tasks.
  - Open problems with respect to architecting agentic AI systems.

1.3 Relevance of Topic for ECSA

More and more industries as well as research communities have started embracing Generative AI as part of their business strategies and software systems.

The topic of this tutorial is highly relevant to ECSA 2025, as it addresses the growing need to understand and evolve software architecture in response to advances in AI—particularly generative and agentic AI. As AI becomes an essential part of modern software systems, software architects are facing new design challenges related to adaptability, coordination, explainability, and system evolution. Furthermore, a substantial share of existing SA knowledge related to design, development, deployment, and evolution can be applied or adapted to Agentic AI systems. The SA community can take advantage of this existing knowledge and identify concrete gaps that require further research and development to support the design of AI-enabled systems. At the same time, AI techniques such as large language models (LLMs) are being used to support architectural tasks, influencing how architecture is practiced in both industry and research. Despite this dual shift, research and experience remain scattered across multiple communities, including software engineering, AI/ML, and software architecture. This tutorial brings these threads together, offering a structured view of the state of the art (in bother research and practice) while identifying open challenges and opportunities for future research. It directly supports the ECSA 2025's focus on impactful architecture and the role of architecting principles in AI-driven systems.

1.4 Intended Target Audience

We expect the tutorial to target several different types of participants:

1. researchers working at the intersection of SA and AI,
2. researchers working on improving architecting practices for AI-enabled systems,
3. researchers working on the adoption of LLMs for SA research,
4. practitioners involved in architecting agentic AI systems,
5. practitioners wanting to explore how to better adopt Generative AI for SA practices.

Concerning the necessary expertise, we believe that any researcher or practitioner with a background in either SA, Generative AI/Machine Learning, or both fields will be able to actively participate in the workshop. The tutorial is structured in a manner to provide adequate background to researchers or practitioners who are also trying to explore this emerging space of software architecture and generative AI.

Considering the increasing use of Generative AI and the rapid evolution of tools/frameworks in this space, the potential interest of the practitioner community, the increasing interest in the research community and the interest displayed by the participants during a working session on "GenAI and Software Architects:

What Skills Will Architects Need in the Age of LLMs?" at ICSA 2025, we expect to have around 35–40 participants in the tutorial.

Tutorial Publicity. We plan to disseminate the Tutorial plan and attract a wider audience by spreading information on the tutorial via:

- the personal SA practitioner and research networks of the tutorial presenters,
- social media groups and channels of SA and ML communities in LinkedIn, Facebook, and Twitter,
- the dedicated social media account of SAML workshop (co-located with ICSA),
- the ECSA, ICSA, ICSE, and ISAPS social networks,
- the list of participants of Dagstuhl Seminar 23302 *Software Architecture and Machine Learning*,

2 Tutorial Implementation

2.1 Duration of the Tutorial

This tutorial is designed as a half-day session with a total duration of 3 h (180 min). It is divided into three main modules, each focusing on a key aspect of the intersection between software architecture and AI: (i) a brief introduction to Software Architecture for Machine Learning (SAML), (ii) Large Language Models for Software Architecture (LLM4SA), and (iii) Architecting Agentic AI Systems (Agentic AI). Each module includes a mix of presentation, interactive discussion, and hands-on activities. Discussions with audience, Q&A, hands-on session and case study based examples are integrated to maintain engagement and allow reflection. Moreover, the participants will also be provided with all the tutorial presentation materials including the slides, source code developed and a repository of agentic AI systems through a dedicated tutorial website.

2.2 Preliminary Schedule

Assuming that the workshop begins at 10 AM, a preliminary schedule is given below:
09:00–09:05: Welcome
09:05–09:25: Session 1, Introducing Software Architecture for ML (SAML)

1. Introduction to basic concepts in ML-enabled systems (10 min)
2. Challenges in architecting AI-enabled systems through a case study (10 min)

09:25–10:00: Session 2, LLMs for Software Architecture (LLM4SA)

1. LLM for Software Architecture (and terminology) (35 min)

10:00–10:30: Coffee break
10:30–11:30: Session 2, LLMs for Software Architecture - continuation (LLM4SA)

1. Demo of a tool, ArchCoPilot (10 min)
2. Discussions with the audience (20 min)
3. Hands-on with ArchCoPilot (20 min)

11:20–12:25: Session 3, Architecting Agentic AI systems

1. Introduction to Agentic AI systems (15 min)
2. Architecting Agentic AI systems (20 min)
3. Case study of Agentic AI framework for autonomous CloudOps (10 min)
4. Hands-on activity for the audience on architecting an agentic AI system (20 min)

12:25–12:30: Tutorial Closing

3 Background of Presenters

The presenters are well-connected in the software architecture community, covering both academia and industry, and have participated in Dagstuhl Seminar 23302 *Software Architecture and Machine Learning*[1], as organizers. Further, the presenters have been the co-organizers of the Software Architecture and Machine Learning Workshop (SAML)[2], which has been co-located with ECSA/ICSA over the last four years.

Karthik Vaidhyanathan is an Assistant Professor at the Software Engineering Research Center, IIIT-Hyderabad, India, where he is also associated with the leadership team of the Smart City Living Lab. He obtained his PhD from the Gran Sasso Science Institute, Italy, and did his postdoc at the University of L'Aquila, Italy. His main research interests lie in the intersection of software architecture and artificial intelligence, specifically focusing on building sustainable and self-adaptive software systems. Over the last two years, he has been working on using Generative AI for improving software architectural practices and further contributed to architecture and development of an open source agentic framework. Karthik also possesses more than five years of industrial experience and serves as a reviewer for various conferences and journals. Karthik is/was also part of the organizing committee of I-Cities 2018, ECSA 2020, SAML 2021, SAML 2022, ICSA 2023, CAIN 2023, CompEd 2023, Dagstuhl Seminar 23302, ICSA 2024, and CAIN 2025. He is also an editorial board member of IEEE Software.

Henry Muccini is a Professor in Computer Science at the University of L'Aquila (Italy). His main research interests are Software Architecture, Model-based/-driven Engineering, and Quality Assurance. He is investigating how ML can be used for better architecting software systems (for supporting architectural self-adaptation, microservices selection and modernization, and green AI), and how to architect ML applications. He is currently highly involved in Agentic AI

[1] https://www.dagstuhl.de/23302.
[2] https://sa-ml.github.io/saml2025.

architectures research. He has been co-organizing the Dagstuhl Seminar 23302, and has been the general co-chair of ICSA 2023, ECSA 2020, I-Cities 2018, and MOBILESoft 2017. He is the SC chair of ICSA, and a member of the ECSA steering committee.

4 Previous Editions

This is the first official edition of this tutorial in its current form. However, the content has been developed and refined over time through various related events/forums. Elements of the tutorial have been presented in invited talks, conferences (such as Conference in AI Engineering - CAIN[3]), workshops (such as SAML workshops), seminars such as the Dagstuhl (23302) seminar on *Software Architecture and Machine Learning* graduate-level course modules, and research group sessions (such as the working session on "What Skills do Architects needs in the age of Generative AI". Moreover, the tutorial will also serve as a bridge between SAML 2025 and the next edition of SAML (SAML 2026) which is being planned to be organized with ICSA 2026.

References

1. Arun, S., Tedla, M., Vaidhyanathan, K.: LLMs for generation of architectural components: an exploratory empirical study in the serverless world. In: 2025 IEEE 22nd International Conference on Software Architecture (ICSA). IEEE (2025)
2. Dhar, R., Vaidhyanathan, K., Varma, V.: Can LLMs generate architectural design decisions? An exploratory empirical study. In: 2024 IEEE 21st International Conference on Software Architecture (ICSA), pp. 79–89. IEEE (2024)
3. Dhar, R., Vaidhyanathan, K., Varma, V.: Leveraging generative AI for architecture knowledge management. In: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp. 163–166. IEEE (2024)
4. Esposito, M., et al.: Generative AI for software architecture. applications, trends, challenges, and future directions. arXiv preprint arXiv:2503.13310 (2025)
5. Forehand, M.: Bloom's taxonomy. Emerg. Perspect. Learn. Teach. Technol. **41**(4), 47–56 (2010)
6. Hou, X., et al.: Large language models for software engineering: a systematic literature review. ACM Trans. Softw. Eng. Methodol. **33**(8), 1–79 (2024)
7. Lewis, G.A., Muccini, H., Ozkaya, I., Vaidhyanathan, K., Weiss, R., Zhu, L.: Software architecture and machine learning (Dagstuhl Seminar 23302). Dagstuhl Rep. **13**(7), 166–188 (2024)
8. Liu, Y., et al.: Agent design pattern catalogue: a collection of architectural patterns for foundation model based agents. J. Syst. Softw. **220**, 112278 (2025)
9. Muccini, H., Vaidhyanathan, K.: Software architecture for ML-based systems: what exists and what lies ahead. In: 2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN), pp. 121–128. IEEE (2021)

[3] https://conf.researchr.org/home/cain-2025.

10. Ozkaya, I.: Application of large language models to software engineering tasks: opportunities, risks, and implications. IEEE Softw. **40**(3), 4–8 (2023)
11. Parthasarathy, K., et al.: Engineering LLM powered multi-agent framework for autonomous CloudOps . In: 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN), pp. 201–211. IEEE Computer Society, Los Alamitos (2025)